Устройство сервера

Go-бэкенд ПОДБОР.про: из чего состоит, как проходит запрос, где правила. Версия после сокращения (октябрь 2026): 997 строк рабочего кода, 12 файлов.

1. Коротко

- Один исполняемый файл: REST-интерфейс `/api/v1` и раздача собранной клиентской части (`web/dist`).
- Договор — `server/api/openapi.yaml`. Он встроен в программу; каждый адрес из него отвечает: либо настоящим обработчиком, либо заглушкой с примером из описания.
- База — PostgreSQL (образ pgvector). Изменения схемы применяются сами при запуске (goose).
- Вход — JWT (HS256) на 24 часа, пароли — bcrypt.
- Сейчас настоящие обработчики у 5 адресов: `GET /health`, `POST /auth/register`, `POST /auth/login`, `GET /me`, `PATCH /me`. Остальные 19 — заглушки.

2. Запуск и настройки

Переменная	Что	По умолчанию
`DATABASE_URL`	адрес PostgreSQL	обязательно
`JWT_SECRET`	секрет подписи токенов, от 32 символов	обязательно
`HTTP_ADDR`	где слушать	`:8080`
`WEB_DIR`	папка со сборкой клиентской части	`./web/dist`

```
docker compose up --build        # вся система: db + ml + server, http://localhost:8080
docker compose up -d db          # только база, для разработки
cd server
make run                         # сервер с настройками для разработки
make test                        # тесты без базы (тесты с базой пропускаются)
make test-db                     # все тесты; ВНИМАНИЕ: очищают таблицу users
make generate                    # типы Go из openapi.yaml (make run/test делают это сами)
go run ./cmd/server -migrate-only   # только применить схему и выйти
```

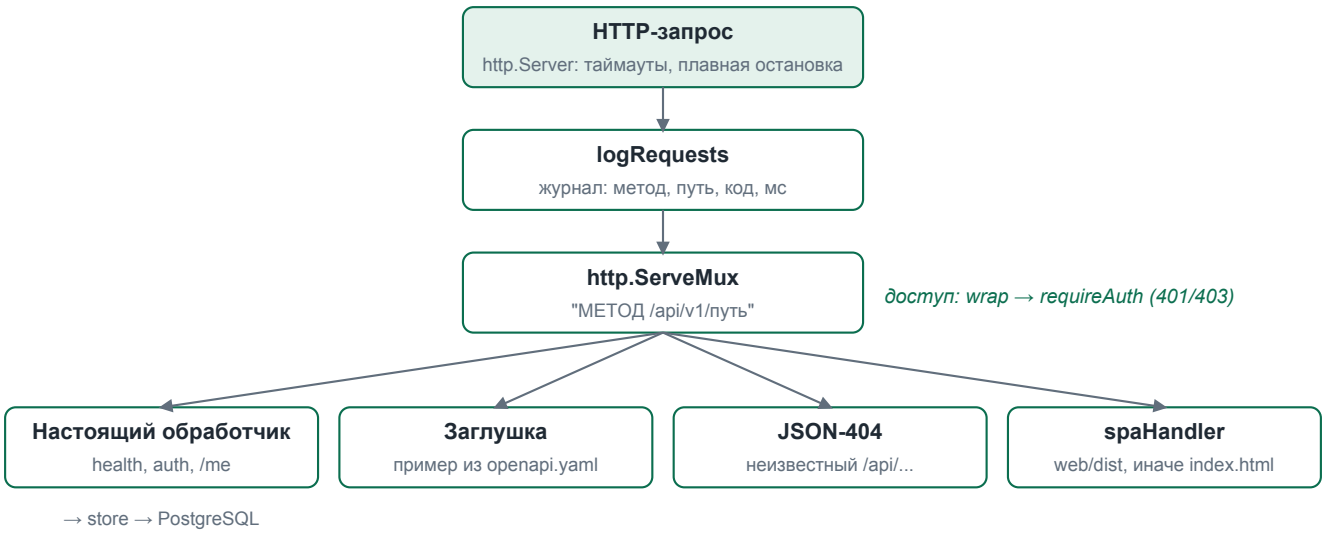
`api/types.gen.go` в git не хранится: его создаёт `make generate` (и Dockerfile, и CI перед сборкой). Свежий клон без него не соберётся, IDE будет подсвечивать ошибки, пока не выполнить `make generate` один раз.

3. Карта файлов

Файл	Строк	За что отвечает
`cmd/server/main.go`	106	Настройки, подключение к базе, миграции, сборка зависимостей, http.Server, остановка по SIGTERM
`api/spec.go`	27	Встраивает openapi.yaml, `Load()` — разбор и проверка описания, `BasePath`
`api/types.gen.go`	—	Типы запросов и ответов из описания (создаётся, не в git)
`migrations/migrations.go`	27	Встроенные *.sql и `Up()` через goose
`internal/httpapi/api.go`	114	Config, маршруты, уровни доступа, `requireAuth`, `UserID(ctx)`
`internal/httpapi/http.go`	112	Единый вид ошибок, `readJSON` с ограничением 1 МБ, раздача SPA, журнал запросов
`internal/httpapi/stubs.go`	81	Заглушки для всех адресов описания без настоящего обработчика
`internal/httpapi/handlers_auth.go`	78	`health`, `register`, `login`
`internal/httpapi/handlers_me.go`	91	`getMe`, `patchMe`, проверка полей профиля
`internal/auth/service.go`	128	Регистрация и вход: проверки, bcrypt, защита от перебора почт
`internal/auth/tokens.go`	80	Выдача и проверка JWT
`internal/store/store.go`	91	Store, ошибки, пользователи (создание, поиск по почте)
`internal/store/profiles.go`	62	Чтение и изменение профиля

Зависимости идут в одну сторону: `main → httpapi → auth → store → api`. Пакет `httpapi` зависит от базы только через интерфейс `Store` (в тестах — подделка).

4. Путь запроса



- Маршрут регистрируется строкой вида `"GET /api/v1/me"` (стандартный ServeMux Go 1.22+): метод и путь проверяет сам маршрутизатор.
- `New()` сначала регистрирует настоящие обработчики (`a.handle`), запоминая их в `a.real`, затем обходит описание и ставит заглушки только туда, где обработчика нет.
- Всё под `/api/v1/`, что не нашлось, — JSON-ошибка `not_found`, а не index.html.
- Остальные пути отдаёт `spaHandler`: существующий файл — как есть, иначе `index.html` с `Cache-Control: no-cache`. Не GET/HEAD — `405 method_not_allowed`.

5. Доступ и токены

Уровень	Как задаётся	Что проверяется
`public`	в коде; для заглушек — `security: []` в описании	ничего
`user`	общий `security` описания	заголовок `Authorization: Bearer <токен>`, подпись, срок → иначе 401
`admin`	`x-admin: true` у операции	то же + признак администратора в токене → иначе 403

- Токен: HS256, поля `sub` (номер пользователя), `adm` (администратор), `iat`, `exp` = +24 ч. Продления нет.
- Проверка разрешает только HS256 (`alg=none` и чужие алгоритмы отклоняются), срок обязателен.
- «Закрыто при сбое»: любая ошибка разбора токена — 401 `unauthorized`.
- Обработчик берёт номер пользователя через `UserID(r.Context())` — права сам не проверяет.
- Признак администратора попадает в токен при входе; если его поменяли в базе, он вступит в силу после нового входа.

6. Регистрация и вход

- Почта приводится к нижнему регистру и проверяется (`net/mail` + точка в домене). В базе — уникальный индекс по `lower(email)`.
- Пароль: от 8 символов, не больше 72 байт (предел bcrypt). bcrypt cost 10.
- `CreateUser` в одной транзакции создаёт пользователя и пустой профиль — профиль есть всегда.
- Неверная почта и неверный пароль дают одинаковый ответ 401 `invalid_credentials`. Для неизвестной почты всё равно выполняется сравнение bcrypt с заглушкой, чтобы время ответа не выдавало, кто зарегистрирован.
- Ответ на успех: `{access_token, token_type: "Bearer", expires_at, user}`; регистрация — 201, вход — 200.

7. Профиль: GET и PATCH /me

- `GET /me` — пользователь и профиль одним запросом (`users JOIN profiles`). Незаполненные `target_title`, `target_grade`, `salary_from` отдаются как `null`. `skills` пока всегда `[]` — таблицы навыков ещё нет.
- `PATCH /me` меняет только переданные поля: `nil` уходит в SQL как NULL, и `COALESCE` оставляет старое значение. Стереть поле в `null` нельзя (так в описании). Изменения `users.full_name` и `profiles` — в одной транзакции. Ответ — профиль после изменения.
- `skill_ids` принимается и пока игнорируется.
- Пользователя нет (удалён, а токен ещё действует) — 404 `not_found`, а не 500.

Поле	Правило (длина — в символах, не байтах)	Текст ошибки
`full_name`	после обрезки пробелов не пустое	Укажите имя
`target_title`	≤ 100	Должность — не длиннее 100 символов
`target_grade`	junior | middle | senior	Уровень: junior, middle или senior
`cities`	≤ 10 штук, каждый ≤ 100	Можно указать не больше 10 городов / Название города — не длиннее 100 символов
`work_formats`	office | remote | hybrid	Формат работы: office, remote или hybrid
`salary_from`	0 … 10 000 000	Зарплата — от 0 до 10 000 000
`tone`	formal | neutral | friendly	Тон: formal, neutral или friendly

Правила — одна таблица в `validatePatchMe`; допустимые значения берутся из констант `api.*`, созданных из описания, поэтому не расходятся с ним. Ошибка — 400 `validation_error` с первым нарушенным правилом.

8. База и правило доступа к данным

Весь SQL — только в `internal/store`. Главное правило:

```
Функция, которая читает или меняет данные пользователя, принимает userID
первым параметром после ctx и ограничивает запрос условием user_id = $N.
Чужая запись выглядит как несуществующая: ErrNotFound -> 404 (не 403).
Исключение: GetUserByEmail при входе (userID ещё неизвестен).
```

Так проверку невозможно забыть: без userID функцию не вызвать, а 404 не выдаёт, что запись существует.

Таблица	Миграция	Назначение
`users`	00001	почта (уникальна без учёта регистра), хеш пароля, имя, `is_admin`
`profiles`	00001	1:1 с users: должность, уровень, города, форматы, зарплата, тон, статус резюме
`jobs`	00002	очередь фоновых задач (collect, normalize, embed, parse_resume) — пока не используется кодом
`sources`	00002	источники вакансий (headhunter)
`collection_runs`	00002	запуски сбора вакансий и их итоги
`vacancy_raw`	00002	сырые ответы источников, уникальны по (источник, внешний id)

Новая миграция — файл `server/migrations/0000N_что.sql` с разделами `-- +goose Up` / `-- +goose Down`. Сервер применит её сам при следующем запуске.

9. Ошибки

Любая ошибка — `{"error": {"code": "...", "message": "..."}}`. `code` — для программы, `message` — для человека, показывается как есть. Подробности ошибок базы пишутся в журнал, наружу уходит общий текст.

HTTP	code	Когда
400	`invalid_json`	тело не разбирается как JSON нужного вида
400	`validation_error`	поле заполнено неверно
401	`unauthorized`	нет токена, он просрочен или подделан
401	`invalid_credentials`	неверная почта или пароль
403	`forbidden`	адрес только для администратора
404	`not_found`	нет записи (или она чужая); неизвестный адрес под /api/
405	`method_not_allowed`	не GET/HEAD к клиентской части
409	`email_taken`	почта уже зарегистрирована
413	`body_too_large`	JSON больше 1 МБ
500	`internal_error`	прочие ошибки (подробности — в журнале)

10. Договор и заглушки

- Заглушка отвечает примером первого успешного ответа (200/201/202/204) и заголовком `X-Stub: true`. Пример кодируется один раз при запуске: ошибка в описании видна сразу, сервер не стартует.
- Доступ у заглушки такой же, какой будет у настоящего обработчика (по `security` и `x-admin`).
- Клиентская часть строит свои типы и заглушки из того же файла (`pnpm run generate` в `web/`).
- Тесты описания (`api/spec_test.go`): файл проходит проверку OpenAPI, в нём есть все 24 обязательных адреса, у каждого ответа с телом есть пример, у каждого адреса перечислены ошибки.

11. Тесты

Файл	Что проверяет	База
`auth/tokens_test.go`	токен туда-обратно; отклоняются просроченный, чужой, изменённый, alg=none; короткий секрет; формат почты	нет
`httpapi/httpapi_test.go`	health; 401 на плохие токены и UserID в обработчике; 403 для admin; все адреса отвечают; пример заглушки; JSON-404; SPA	нет
`httpapi/auth_db_test.go`	регистрация, вход, одинаковый ответ для неизвестной почты, пустой профиль; PATCH /me: частичное изменение, изоляция пользователей, все ошибки проверки, русские названия; 404 для удалённого пользователя	да
`api/spec_test.go`	описание целое и полное (см. п. 10)	нет

Тесты с базой запускаются при заданном `TEST_DATABASE_URL` (в CI — всегда) и очищают таблицу `users`. Не направляйте их на базу с нужными данными.

CI (`.github/workflows/server.yml`): `make generate` → gofmt → go vet → миграции на пустой базе → `go test -race ./...`.

12. Как добавить настоящий обработчик

1. Убедиться, что адрес и схемы есть в `openapi.yaml`; если меняли описание — `make generate`.
2. SQL — новая функция в `internal/store` с `userID` первым параметром и `user_id = $N` в запросе.
3. Добавить метод в интерфейс `Store` (`httpapi/api.go`) и в подделку `fakeStore` в тестах.
4. Обработчик — метод `*API`: `readJSON` → проверка → вызов store с `UserID(ctx)` → `writeJSON`; ошибки store — через `a.storeError`.
5. Одна строка в `New()`: `a.handle("GET", "/vacancies", user, a.listVacancies)`. Заглушка исчезнет сама.
6. Тест с базой: успех, ошибки проверки и то, что чужие данные не видны (404).

13. Что изменилось при сокращении

	Было	Стало
Рабочий код (руками)	1004 строки, 22 файла	997 строк, 12 файлов
Генерированный код в git	881	0 (создаётся при сборке)
`back_jr/` (не собирался)	214	встроен: `GET/PATCH /me` работают
Дубликат `.github/workflows/Validation.go`	53	удалён
Пустые пакеты (`doc.go`)	9	удалены
Тесты	559	633 (добавлены тесты /me)
Настоящих адресов	3	5

При переносе `back_jr` исправлено: `int` → `int64`; 404 вместо 500 для отсутствующего пользователя; `null` вместо `""/0`; обязательное `skills`; `full_name` в PATCH; ограничение размера тела и код `invalid_json`; длина в символах; верный текст ошибки для города; проверка таблицей правил.